# CS151 Data Structures

**Quiz: 3** **Date: 10/1/26**
**Name:**

---

1. Design a Java class `Book` that implements `Comparable` and provides lexicographical (dictionary) odering based on the book's title.

2. Modify your `Book` class so that the ordering is now by price in descending order.

3. 
```
public interface StringChecker {
  /** Returns true if str is valid. */
  boolean isValid(String str);
}
```

   A `StringChecker` interface describes classes that check if strings are valid, according to some criterion. A `CodeWordChecker` is a `StringChecker`. A `CodeWordChecker` object can be constructed with three parameters: two integers and a string. The first two parameters specify the min and max code word lengths, respectively, and the third parameter specifies a string that must NOT occur in the code word. A `CodeWordChecker` object can also be constructed with a single parameter that specifies a string that must NOT occur; in this case the min and max lengths will default to 6 and 20.

   The following examples illustrate the behavoir of `CodeWordChecker` objects.

   Examples

```
StringChecker sc1 = new CodeWordChecker(5, 8, "$");
StringChecker sc2 = new CodeWordChecker("pass");
```

| Method call | Return Value | Explanation |
|---|---|---|
| `sc1.isValid("happy")` | `true` | The code word is valid. |
| `sc1.isValid("happy$")` | `false` | The code word contains `"$"` |
| `sc1.isValid("abc")` | `false` | The code word is too short. |
| `sc1.isValid("happyCode")` | `false` | The code word is too long. |
| `sc2.isValid("MyPass")` | `true` | The code word is valid. |
| `sc2.isValid("Mypassword")` | `false` | The code word contains `"pass"`. |
| `sc2.isValid("happy")` | `false` | The code word is too short. |
| `sc2.isValid("abcdefghijklmnopqrstuvwxyz")` | `false` | The code word is too long. |

Write the complete `CodeWordChecker` class. Your implementation must meet all specifications and conform to all examples.